

Author: Gu Shucheng

Student ID: U2220288L

Topic2: Personal Assistant-as-a-Service

Codebase: https://github.com/ParrotG/learn_pilot

LearnPilot: A Conversation-Centric Personal Assistant-as-a-Service for Academic Workflows

1. Project Overview

LearnPilot is a web-based personal assistant system developed for Topic 2: Personal Assistant-as-a-Service. The project targets academic document workflows and transforms uploaded study materials into persistent, assistant-supported artifacts that can be queried, refined, exported, and synchronized with external productivity tools. The implemented system provides a conversation-centric assistant workspace built on a FastAPI backend and a Next.js frontend.

The completed platform allows a user to upload PDF documents into a chat session, ask questions about the attached material, request summaries, generate and revise study notes, extract date-related tasks, create approved Google Calendar events, export notes into office-document formats, and upload exported artifacts to Google Drive. These actions are coordinated through persistent conversations, explicit tool calls, approval checkpoints, and stored assistant-run state.

2. Problem Statement and Motivation

Students frequently receive course syllabi, assignment briefs, lecture notes, and announcements in PDF form. These documents contain key academic information, but extracting useful knowledge from them usually requires several disconnected steps. A student may need to read the document carefully, identify important points, copy deadlines into a calendar, rewrite the content into revision notes, and then convert those notes into a format suitable for sharing or presentation. Existing document viewers provide storage and reading functions, while general-purpose LLM interfaces provide text interaction. Academic workflow support still requires additional coordination across several tools and several manual steps.

LearnPilot addresses this gap by combining document understanding, multi-turn assistant interaction, note generation, calendar support, document export, and cloud upload in a single assistant platform. The system acts as software that coordinates an academic workflow across several services and preserves the resulting outputs as durable artifacts.

3. Objectives

This project aims to create a modular **Personal Assistant-as-a-Service** system that uses LLM-based conversation as the main entry point and uses tool calling to support integrated academic task management and assistance. The

intended effect is a unified assistant experience in which a user can interact through natural-language dialogue while the system manages documents, notes, dates, exports, and cloud backups within one continuous workflow.

4. System Architecture

The implemented architecture is organized into four main layers: the frontend application, the backend API layer, the assistant runtime layer, and the persistence layer together with external integrations. This layered organization separates user interaction, business logic, run orchestration, storage, and third-party service access.

The frontend is implemented with Next.js, React, TypeScript, and Tailwind CSS. It provides the user-facing workspace, including the chat interface, sidebar navigation, dashboard, settings page, session note panel, artifact panel, and tool approval modal. The user interacts with the frontend, which sends requests through a same-origin proxy to the backend API.

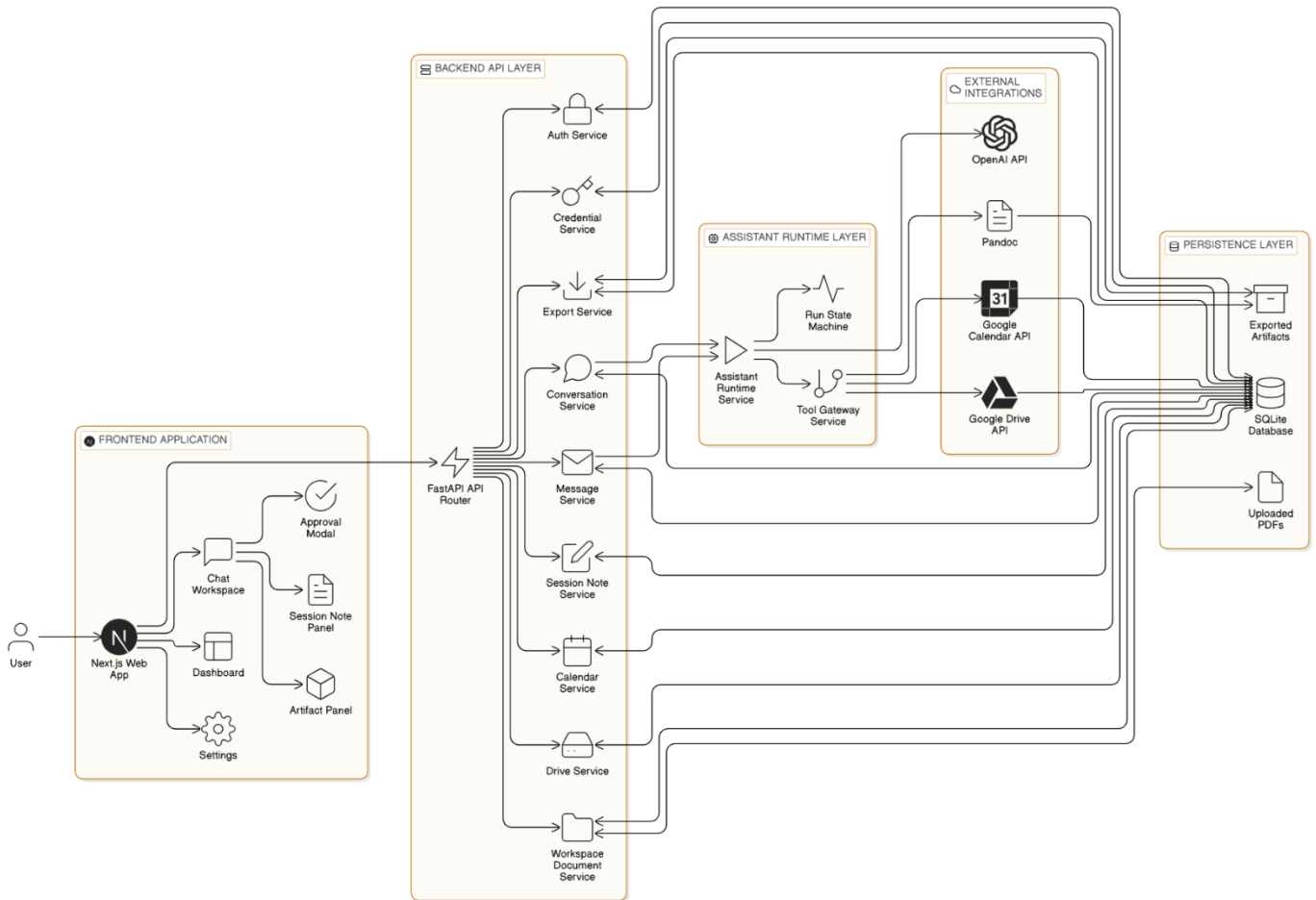
The backend is implemented with FastAPI, SQLAlchemy Async, Pydantic, and Alembic. It exposes RESTful endpoints for authentication, credential management, conversation and message handling, document upload, assistant runs, session-note access, export-artifact access, tool approvals, and Google Drive upload requests. These APIs provide the communication surface required by the project brief.

The assistant runtime layer processes requests through a run-based state machine. A request may produce either a direct assistant reply or a structured tool request. When a requested tool has an external side effect, the run pauses and enters a waiting-for-approval state. Once approved or rejected, the run resumes and persists the resulting state. This design makes the assistant traceable and keeps external actions controlled.

The persistence layer combines SQLite with filesystem storage. Structured application data such as users, conversations, messages, notes, tool calls, approvals, and export artifacts are stored in the database, while uploaded PDFs and generated export files are stored on disk. OpenAI, Google Calendar, Google Drive, and Pandoc are integrated around this core platform.

Because the project involves multiple subsystems with non-trivial runtime dependencies, a Docker-based environment was also provided to offer reproducible configuration, dependency setup, and service startup.

Figure 1 System architecture diagram



5. External Tools and Integrations

Several external tools are essential to the implemented system because they extend the assistant from conversational reasoning to practical academic workflow support.

- OpenAI is one of the most widely used LLM providers. It provides strong document understanding, intent recognition, and tool-request generation capabilities, so it is suitable for acting as the language-model provider in this project. In the implementation, it is configured through a user-provided API key and invoked during assistant-run processing for reply generation, tool selection, and post-tool follow-up responses.
- Google Calendar is a widely used calendar platform that provides event-creation and scheduling APIs. It provides persistent calendar-event management, so it is suitable for handling extracted deadlines and academic dates in this project. In the implementation, it is configured through Google OAuth credentials and invoked after user approval to create calendar events from assistant-generated candidate dates.
- Google Drive is a cloud file-storage platform that provides file-upload and metadata APIs. It provides remote storage and cross-device access, so it is suitable for backup and artifact management in this project. In the implementation, it is configured through Google OAuth and invoked to archive uploaded PDF documents and to upload generated export artifacts to a LearnPilot-managed Drive folder.
- Pandoc is a widely used document-conversion tool. It provides reliable format conversion from Markdown to office-document formats, so it is suitable for note export in this project. In the implementation, it is configured

as a host or container binary through PANDOC_BINARY and invoked by the export service to generate docx and pptx outputs from session-note Markdown.

6. Backend Implementation

According to the project requirement, the capabilities of the personal assistant should be decomposed into independent services that communicate through well-defined RESTful APIs. This requirement is reflected directly in the backend design of LearnPilot. The platform is organized around route modules that expose dedicated HTTP endpoints for each major area of functionality. The Swagger interface at localhost:8000/docs groups these endpoints into the following modules.

6.1 auth

- POST /api/auth/register: registers a new user account.
- POST /api/auth/login: authenticates a user and returns an access token.
- GET /api/auth/me: returns the authenticated user profile.
- PATCH /api/auth/me: updates editable profile fields for the authenticated user.

6.2 credentials

- POST /api/credentials/llm: stores or updates the user's LLM credential configuration.
- POST /api/credentials/google/connect: starts the Google OAuth connection flow.
- GET /api/credentials/google/callback: completes the Google OAuth callback and stores Google credentials.
- GET /api/credentials/status: returns the current status of configured LLM and Google credentials.

6.3 conversations

- POST /api/conversations: creates a new conversation.
- GET /api/conversations: lists the user's conversations.
- GET /api/conversations/{conversation_id}: returns detailed metadata for a single conversation.
- GET /api/conversations/{conversation_id}/note: returns the session note associated with the conversation.
- DELETE /api/conversations/{conversation_id}: deletes a conversation.

6.4 messages

- GET /api/conversations/{conversation_id}/messages: lists all messages in a conversation.
- POST /api/conversations/{conversation_id}/messages: creates a new message and may also trigger an assistant run.

6.5 workspace

- GET /api/conversations/{conversation_id}/documents: lists documents attached to a conversation.
- POST /api/conversations/{conversation_id}/documents: uploads a PDF and attaches it to the conversation.

6.6 runs

- POST /api/conversations/{conversation_id}/runs: creates a new assistant run for a conversation.
- GET /api/runs/{run_id}: retrieves the current state of an assistant run.

6.7 tool-calls

- GET /api/tool-calls/{tool_call_id}: retrieves a tool-call record and its current status.
- POST /api/tool-calls/{tool_call_id}/approve: approves a pending tool call and resumes the run.
- POST /api/tool-calls/{tool_call_id}/reject: rejects a pending tool call and resumes the run with rejection state.

6.8 documents

- POST /api/documents/upload: uploads a PDF to the workspace through the document-centric route group.
- GET /api/documents: lists stored workspace documents.
- GET /api/documents/{document_id}: returns detailed metadata and extracted content for a document.
- POST /api/documents/{document_id}/analyze: performs document-centric analysis through the earlier analysis flow.

6.9 assistant

- POST /api/assistant/execute: executes the earlier assistant-orchestrator analysis flow for a selected document.

6.10 notes

- GET /api/notes: lists session notes.
- GET /api/notes/{note_id}/revisions: returns revision history for a session note.
- POST /api/notes/save: saves a document-centric generated note through the earlier note flow.

6.11 calendar

- POST /api/calendar/extract-events: extracts candidate events from a document.
- POST /api/calendar/create-events: creates calendar events from selected candidate events.

6.12 drive

- POST /api/drive/archive: uploads an original document to Google Drive archive storage.
- GET /api/drive/files/{document_id}: retrieves Drive archival metadata for a document.
- POST /api/drive/upload-artifact: creates a Drive-upload request for an exported artifact.

6.13 exports

- GET /api/conversations/{conversation_id}/exports: lists export artifacts for a conversation.
- POST /api/conversations/{conversation_id}/exports: creates a new export request for the conversation note.
- GET /api/exports: lists recent export artifacts.
- GET /api/exports/{artifact_id}: retrieves metadata for a specific export artifact.
- GET /api/exports/{artifact_id}/download: downloads the generated export artifact file.

6.14 default

- GET /health: returns service health information and runtime dependency status.

Together, these route groups form the RESTful communication surface of the platform. Authentication, conversation management, note access, run orchestration, export handling, and cloud synchronization are all exposed through

explicit HTTP modules. This structure realizes the idea of a modular PAaaS backend through cooperating services and defined API boundaries.

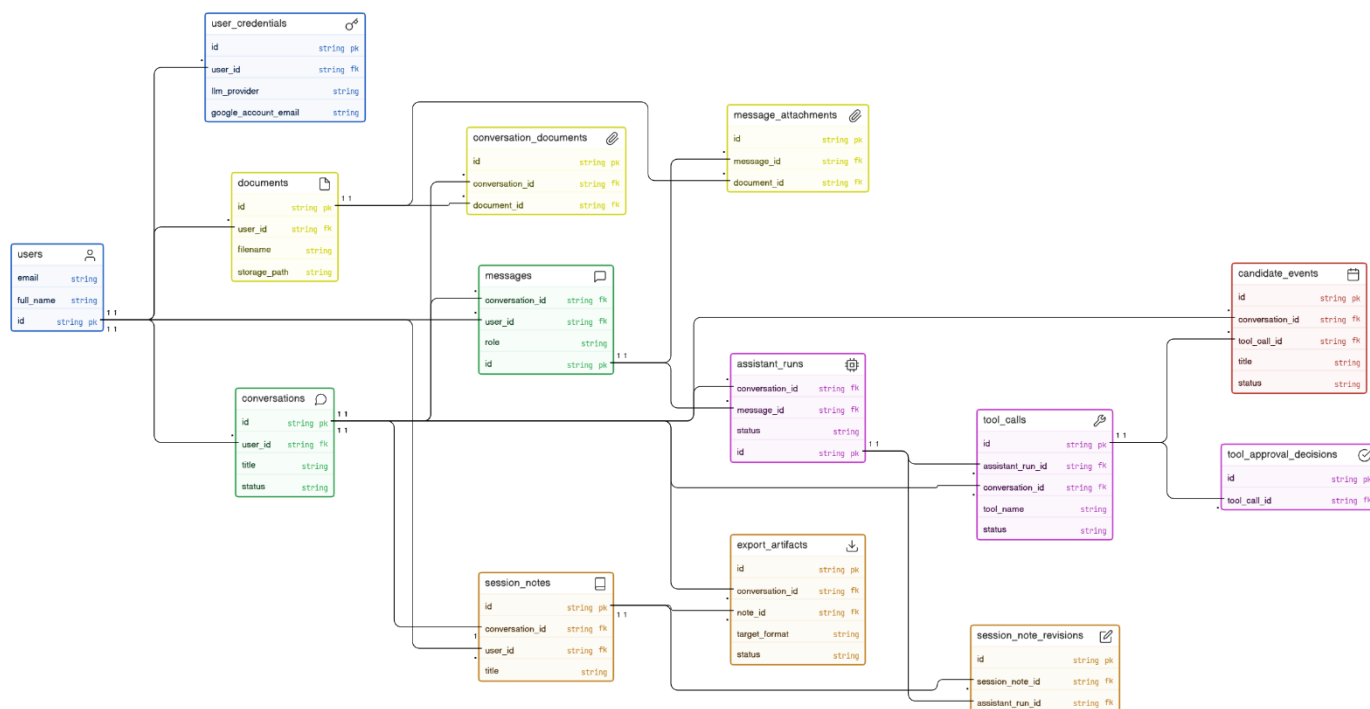
7. Database

SQLite is a lightweight relational database engine that stores all application data in a local file and requires minimal infrastructure for development and demonstration. It was adopted in this project because it is easy to configure, integrates well with SQLAlchemy and Alembic, and provides sufficient persistence for a coursework-scale assistant platform.

In the implemented system, SQLite persists user accounts, encrypted credential metadata, uploaded document metadata, extracted text references, conversation state, messages, assistant runs, tool calls, approval decisions, session notes, note revisions, candidate calendar events, calendar synchronization records, and export-artifact metadata. This persistence layer is critical because the platform depends on cross-turn memory, approval resumption, revision history, and artifact tracking.

The most important persistent entities and their relations are shown below.

Figure 2 Database ER diagram



8. Frontend Implementation

The frontend is centered on the chat page as the main workspace. The layout includes a sidebar with navigation and conversation history, a central message timeline, a composer positioned at the bottom for text input and document upload, and a right-side panel that displays the current session note together with generated export artifacts. This arrangement reflects the intended workflow: the conversation is the main working surface, while notes and generated files remain visible as persistent structured outputs.

Assistant messages and session notes are rendered as Markdown. This improves readability because study-oriented

outputs frequently benefit from headings, bullet structures, emphasis, and compact note formatting. The session note panel presents the latest note as a persistent assistant-managed artifact. The artifact panel presents generated files together with their format, status, creation time, and Drive upload state. Download and upload actions are attached directly to these artifacts so that the user can continue the workflow without leaving the conversation page.

The approval modal provides a unified interface for approval-required tool calls. The modal adapts its content according to the tool type while preserving the same overall approval model. This design keeps the user experience coherent and communicates that the assistant is interacting with external tools in a controlled manner.

9. Example User Workflow

The following example illustrates a complete end-to-end interaction with the implemented system. It is written as a concrete usage scenario.

Step 1. Account Creation and Login

The workflow begins by creating an account and signing into the system. Once authenticated, the user gains access to the protected workspace, including the dashboard, chat page, and settings page.

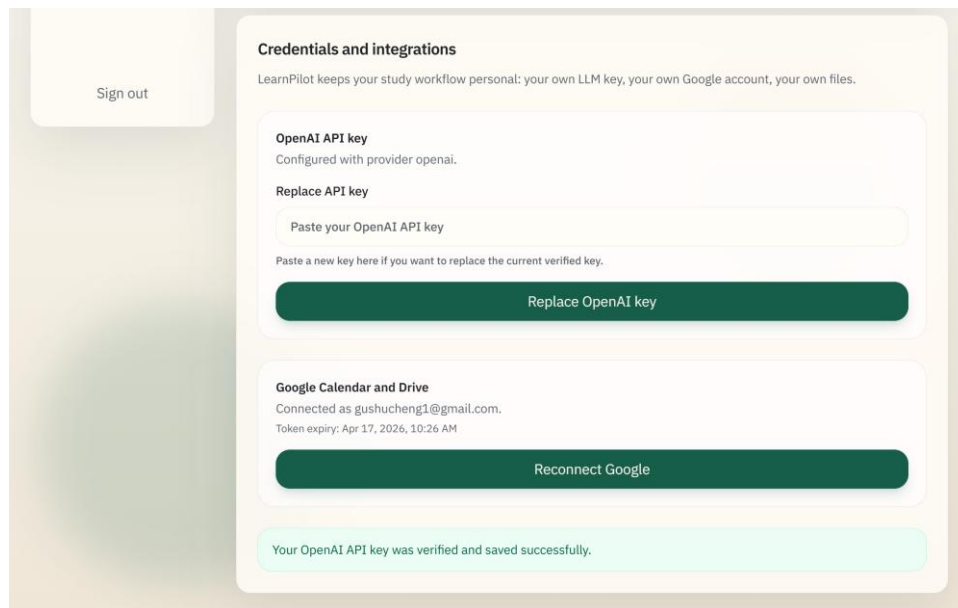
The image displays two screenshots of a web application interface for creating and logging into a workspace.

The top screenshot shows the "Create your LearnPilot workspace" form. It includes a sub-header "Set up your account to upload study PDFs, generate notes, and sync important deadlines." The form fields are: "Full name" (GuShucheng), "Email address" (gushucheng1@gmail.com), and "Password" (masked with dots). A green "Create account" button is at the bottom, with a link "Already have an account? Sign in here" below it.

The bottom screenshot shows the "Welcome back" login form. It includes a sub-header "Sign in to continue working with your documents, notes, and schedule suggestions." The form fields are: "Email address" (gushucheng1@gmail.com) and "Password" (masked with dots). A green "Sign in" button is at the bottom, with a link "New to LearnPilot? Create one now" below it.

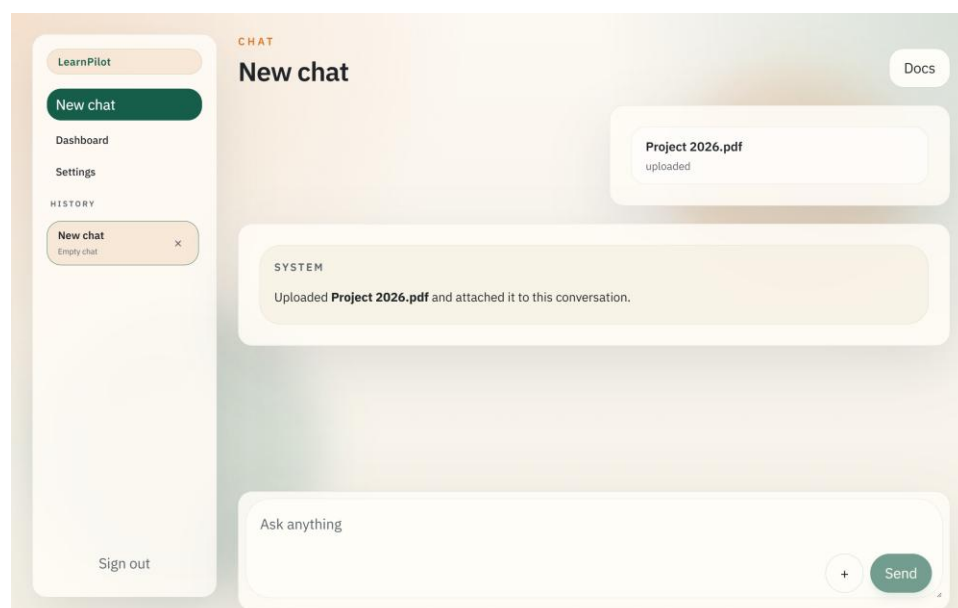
Step 2. LLM and Google Configuration in Settings

After login, the Settings page is used to configure the OpenAI API key and connect the Google account through OAuth. This step is necessary because assistant reasoning and external actions depend on per-user credentials.



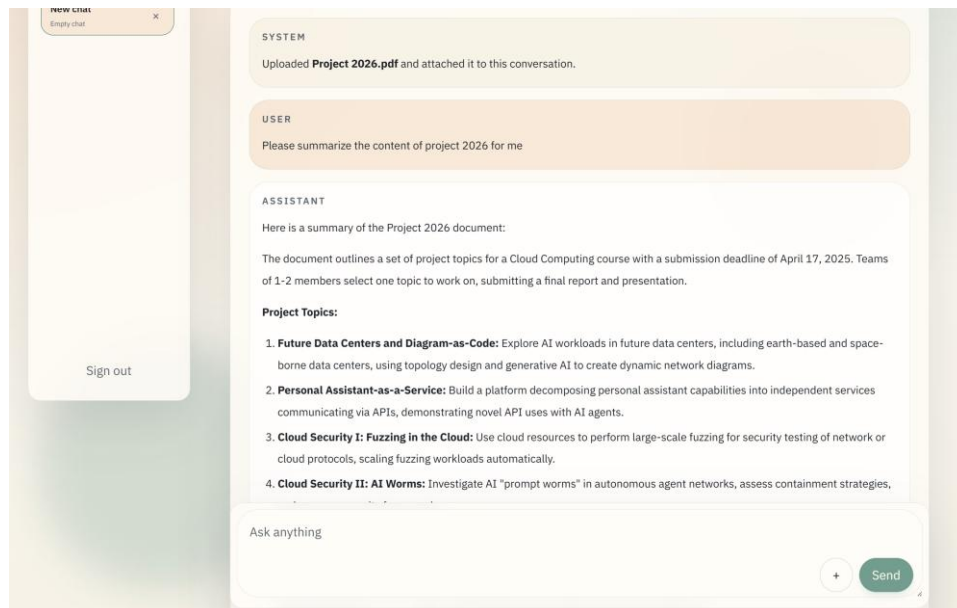
Step 3. PDF Upload in a New Chat

A new conversation is created from the chat workspace, and a PDF document is uploaded directly within that conversation. The uploaded document is stored in the workspace, linked to the session, and made available to the assistant as contextual input.



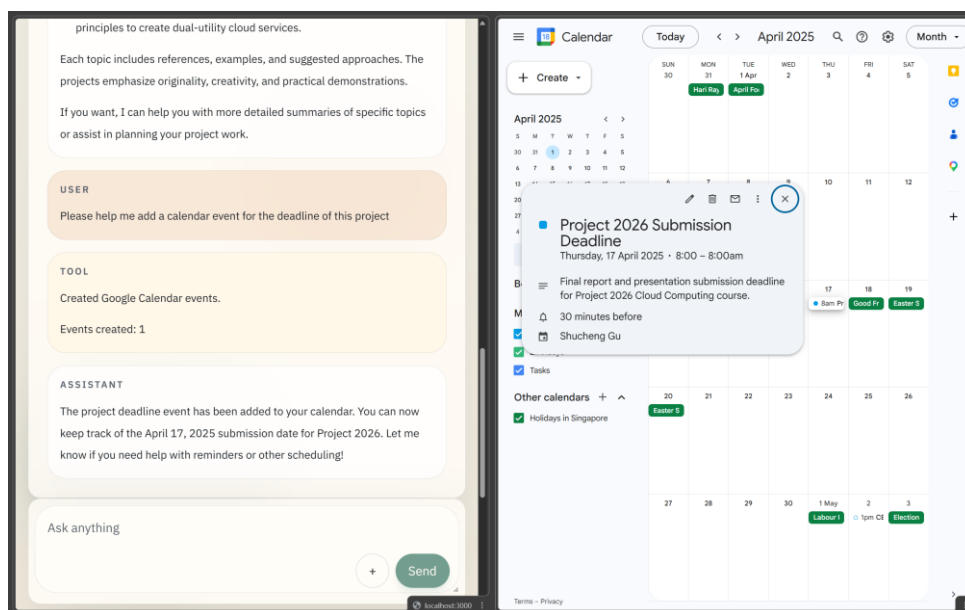
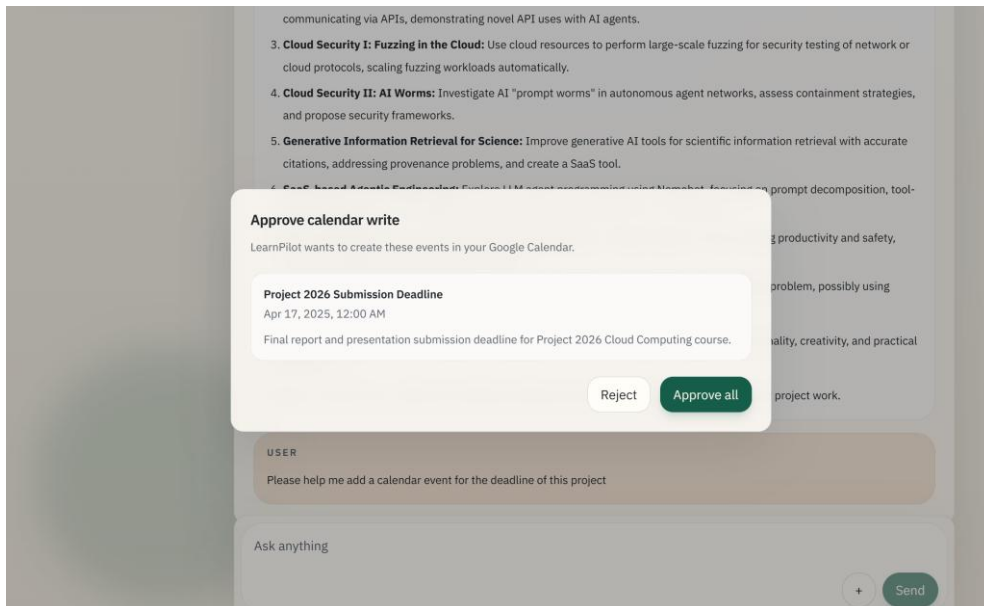
Step 4. Requesting a PDF Summary

The user then asks the assistant to summarize the uploaded PDF. The assistant reads the conversation context together with the attached document excerpt and returns a Markdown-formatted summary in the message timeline.



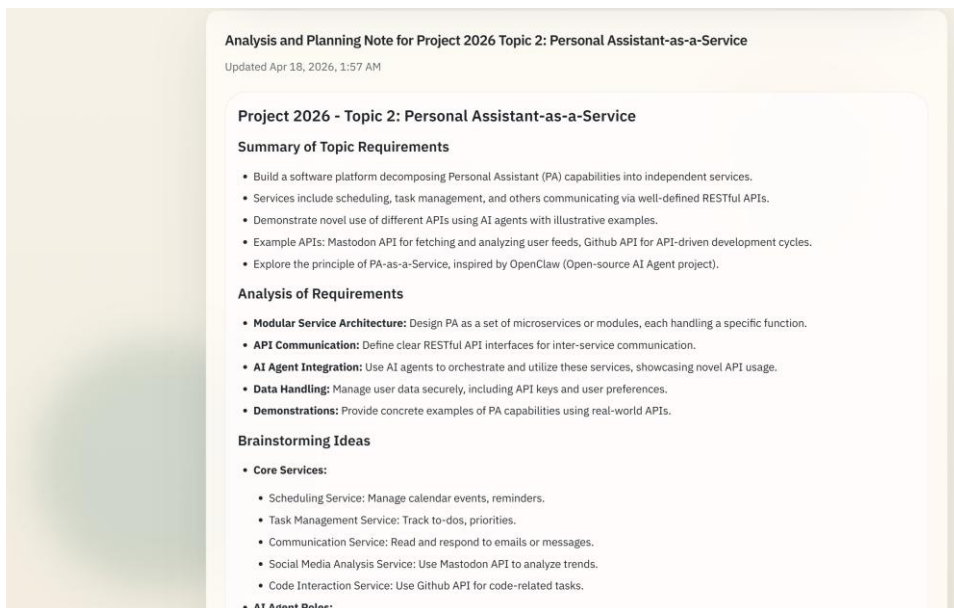
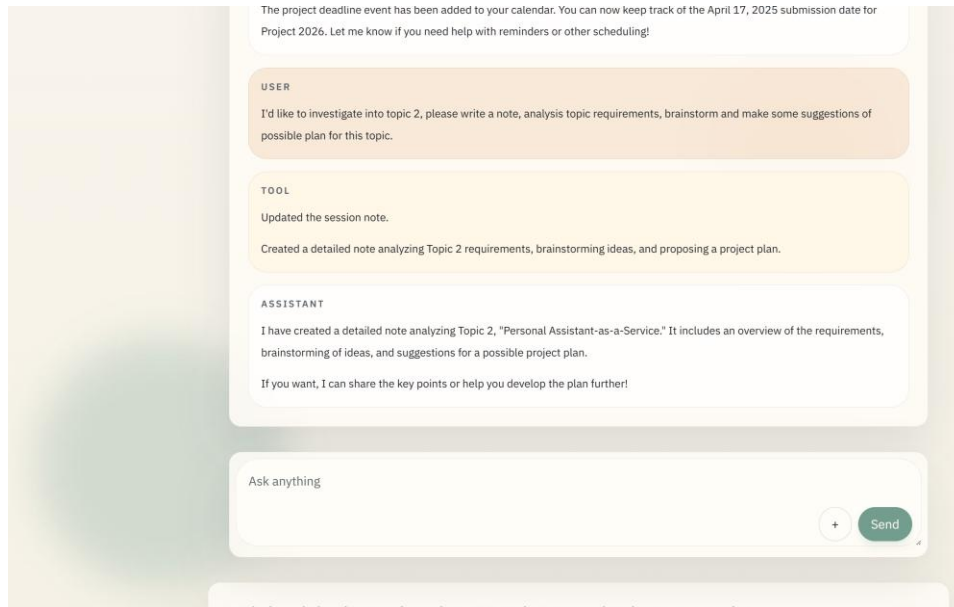
Step 5. Extracting Key Dates and Creating Calendar Events

Next, the user asks the assistant to identify important dates in the PDF and create corresponding Google Calendar events. The assistant extracts candidate events, pauses for approval, and displays the request through the tool-approval modal. After approval, the events are written to Google Calendar and the result is persisted in the conversation history.



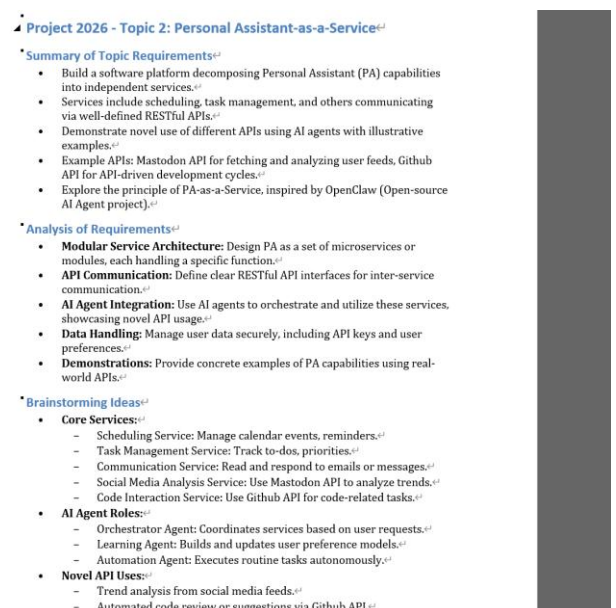
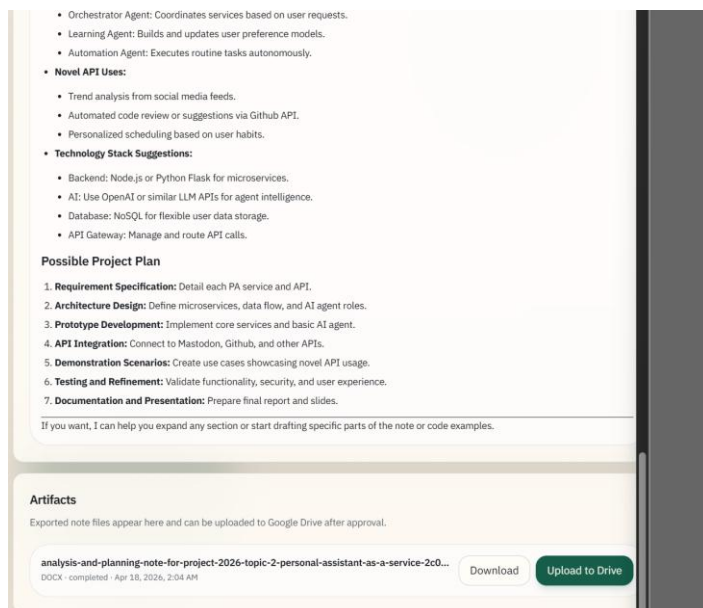
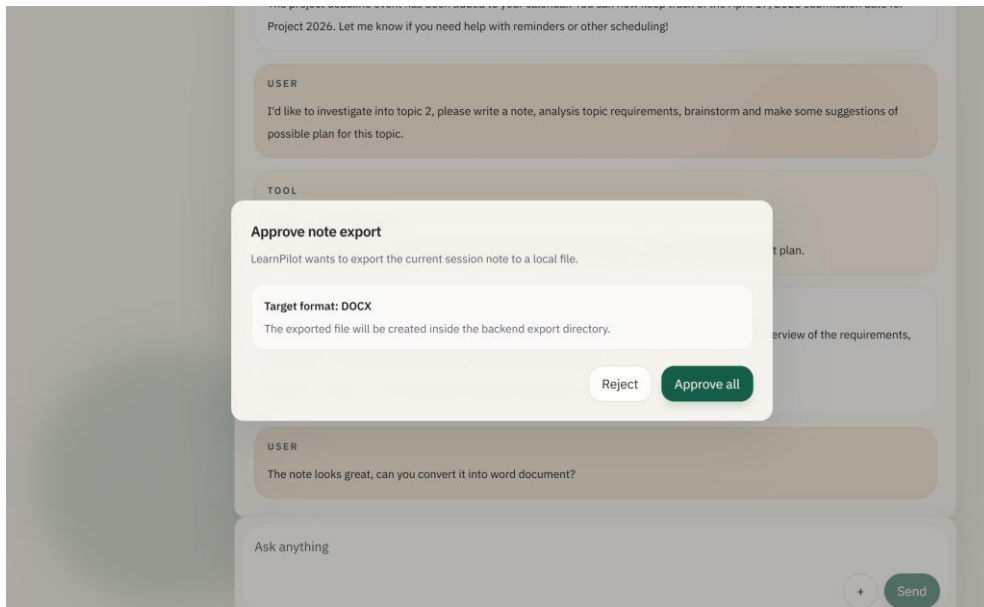
Step 6. Generating a Study Note

The user then asks the assistant to generate a note from the document according to a study-oriented instruction. The assistant creates or updates the session note, and the resulting note appears in the right-side note panel as a persistent Markdown artifact.



Step 7. Converting the Note to DOCX

After the note is generated, the user requests that it be converted into docx format. The assistant creates an export tool request, the system asks for approval, and once approved, Pandoc generates the office-document artifact. The generated docx file then appears in the artifact panel.



Step 8. Generating a Presentation-Oriented Note

The workflow continues with a second instruction that asks the assistant to produce a more concise note suitable for presentation slides, including explicit page-break markers and a shorter, presentation-friendly structure. The assistant updates the session note accordingly so that the content becomes suitable for subsequent slide export.

Analysis and Planning Note for Project 2026 Topic 2: Personal Assistant-as-a-Service (Slide Format)
Updated Apr 18, 2026, 2:14 AM

Project 2026 - Topic 2: Personal Assistant-as-a-Service

Summary of Topic Requirements

- Build a software platform decomposing Personal Assistant (PA) capabilities into independent services.
- Services include scheduling, task management, and others communicating via well-defined RESTful APIs.
- Demonstrate novel use of different APIs using AI agents with illustrative examples.
- Example APIs: Mastodon API for fetching and analyzing user feeds, Github API for API-driven development cycles.
- Explore the principle of PA-as-a-Service, inspired by OpenClaw (Open-source AI Agent project).

Analysis of Requirements

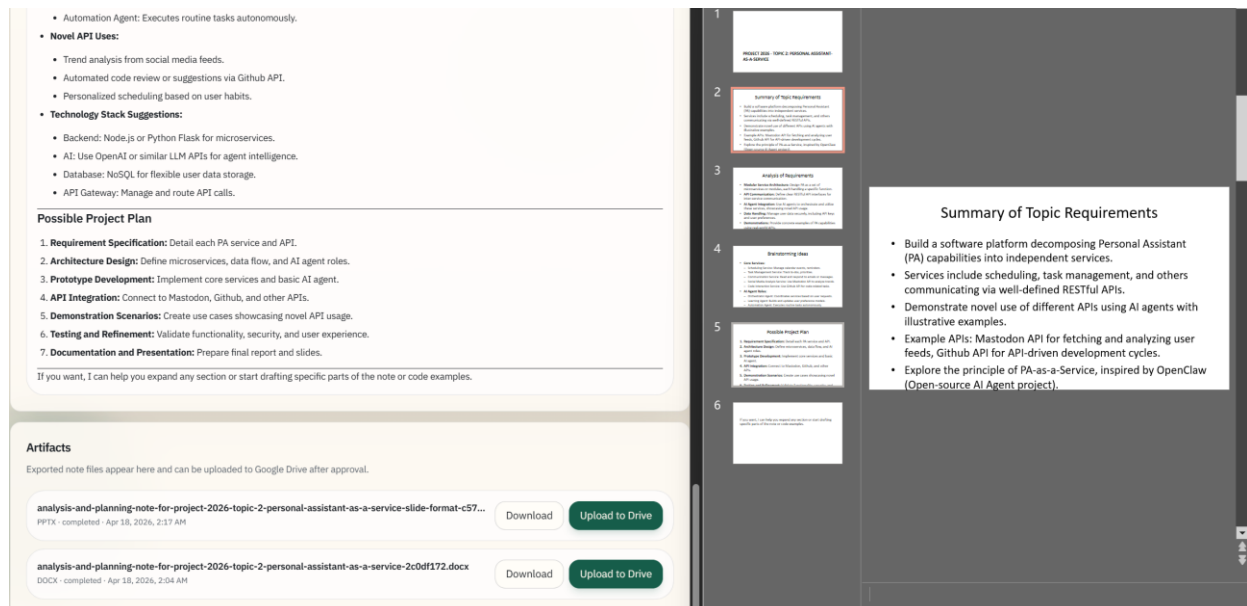
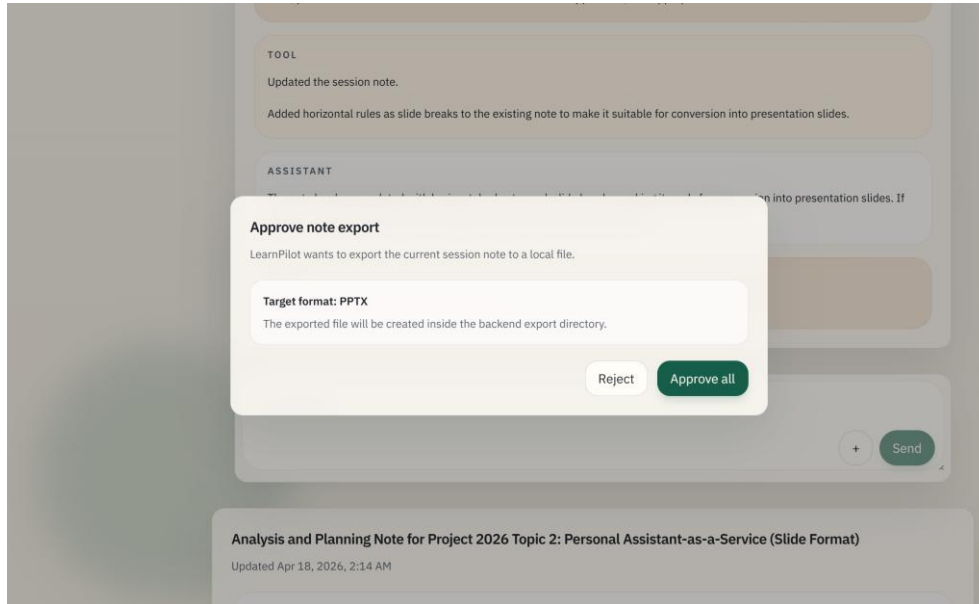
- **Modular Service Architecture:** Design PA as a set of microservices or modules, each handling a specific function.
- **API Communication:** Define clear RESTful API interfaces for inter-service communication.
- **AI Agent Integration:** Use AI agents to orchestrate and utilize these services, showcasing novel API usage.
- **Data Handling:** Manage user data securely, including API keys and user preferences.
- **Demonstrations:** Provide concrete examples of PA capabilities using real-world APIs.

Brainstorming Ideas

- **Core Services:**
 - Scheduling Service: Manage calendar events, reminders.
 - Task Management Service: Track to-dos, priorities.
 - Communication Service: Read and respond to emails or messages.
 - Social Media Analysis Service: Use Mastodon API to analyze trends.
 - Code Interaction Service: Use Github API for code-related tasks.
- **AI Agent Roles:**

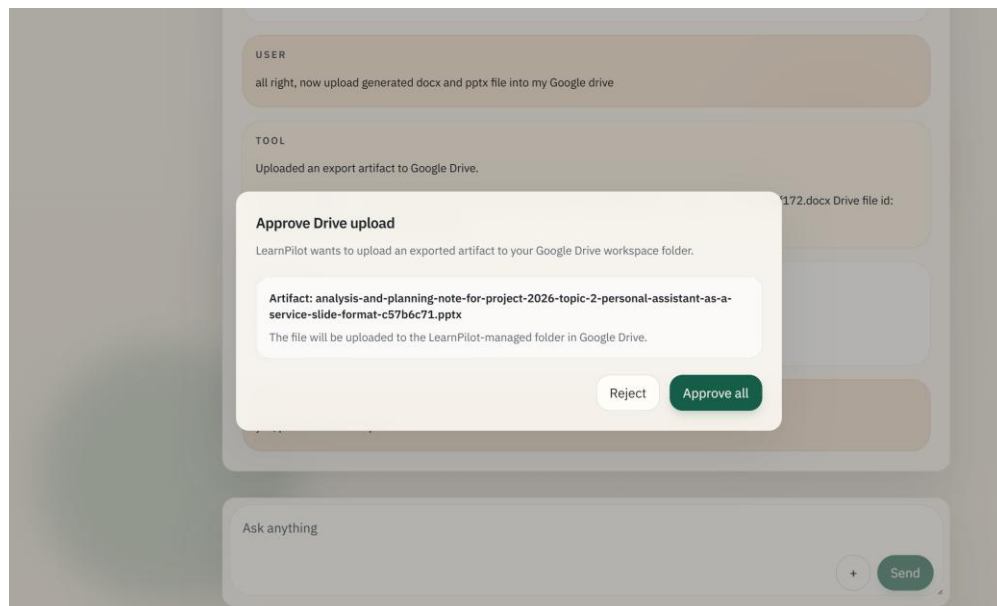
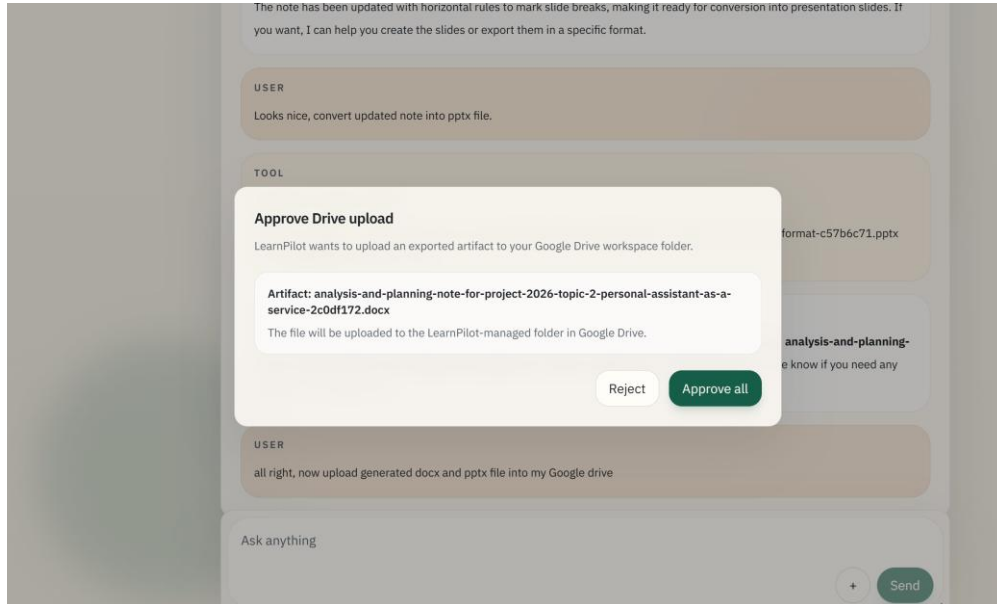
Step 9. Converting the Slide-Style Note to PPTX

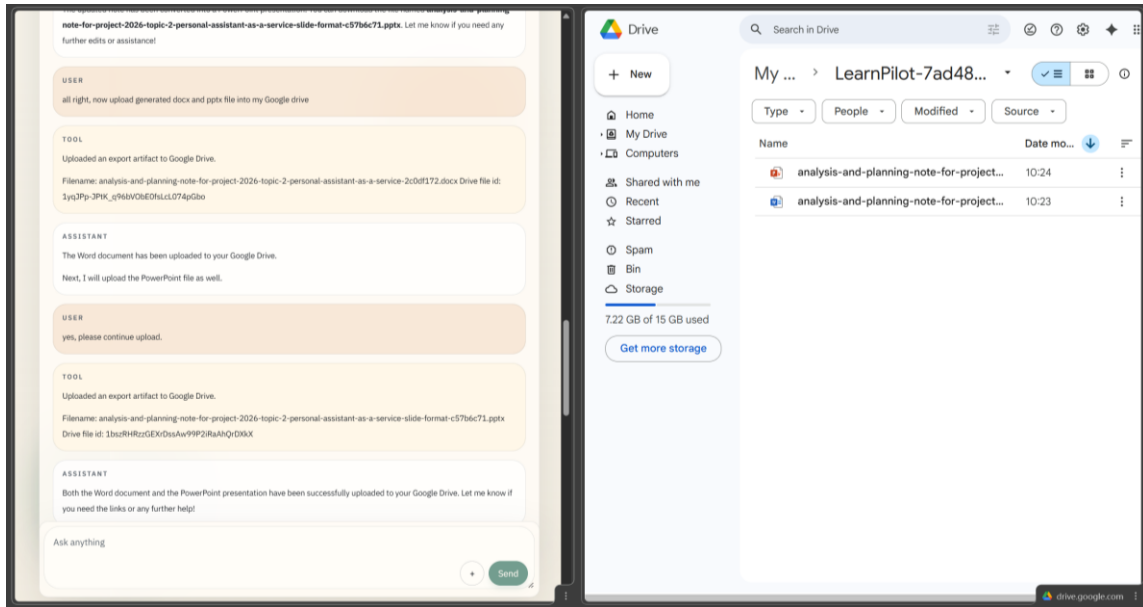
Once the note has been rewritten in a slide-friendly structure, the user requests conversion to pptx. As with the docx export, the assistant requests approval, the export is executed through Pandoc, and the resulting presentation file appears as a second artifact in the artifact panel.



Step 10. Uploading Both Notes to Google Drive

Finally, the user requests backup of both generated artifacts to Google Drive. Each upload request is approval-controlled, and after approval the files are written into the app-managed Drive folder. The artifact panel then reflects the uploaded state for both files.





10. Conclusion

LearnPilot demonstrates a working personal assistant platform for academic workflows and combines persistent chat sessions, session-level note generation, approval-aware external actions, office-document export, and cloud synchronization support. The implemented system satisfies the central idea of a Personal Assistant-as-a-Service platform by exposing assistant capabilities through modular backend services, persistent state, and controlled integrations with external tools.

The resulting platform is coherent, testable, and demonstrable as a coursework submission. Several practical improvements remain possible. Future enhancement could focus on OCR-based recognition methods for scanned inputs, support for a wider range of input and output file formats, more asynchronous API execution so that a greater number of tasks can proceed concurrently, and template-based formatting support for docx and pptx generation.